SIT707 - Selenium Test Cases(2.1P)

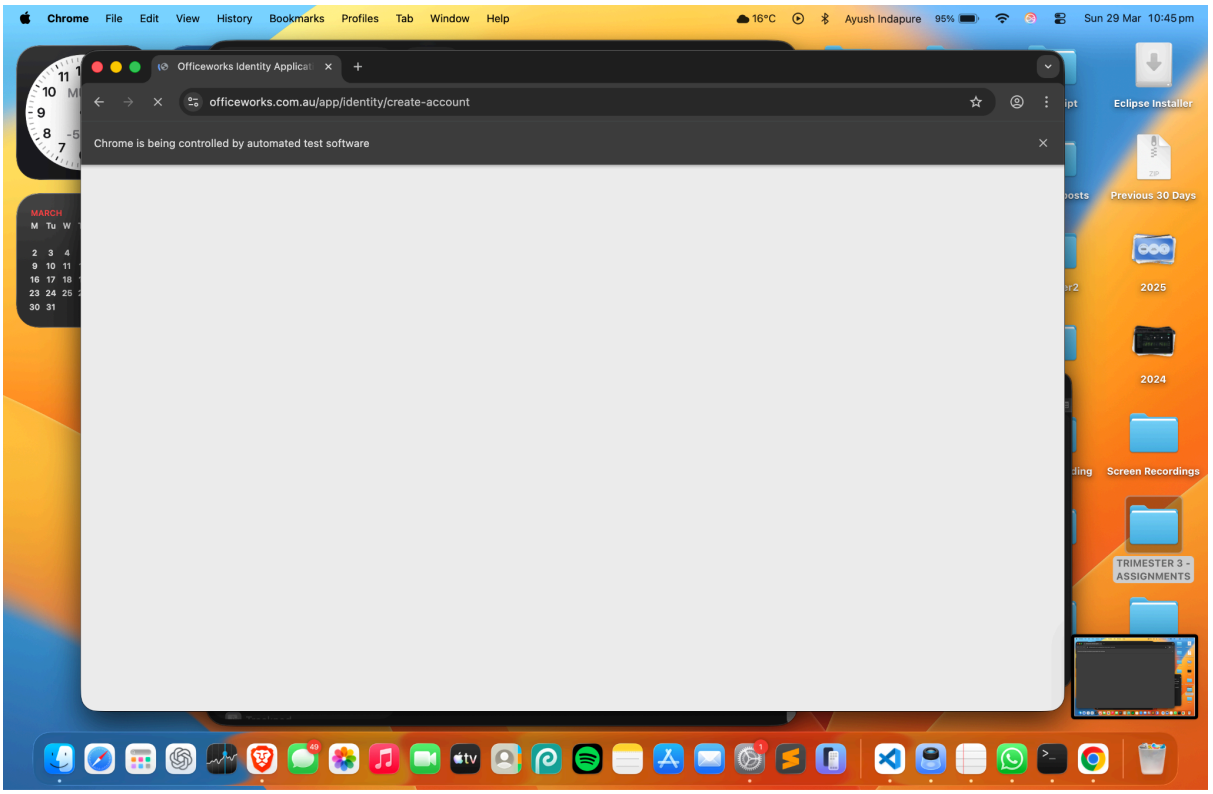
Name : Ayush Indapure
Student ID : 224880003

1. Introduction

In this task, I have used Selenium WebDriver(by downloading and manipulating the path afterwards) to automate the testing of registration pages. Selenium is a browser automation tool which allows us to simulate real user actions like typing, clicking, and submitting forms.

The main objective of this task is to understand how to locate HTML elements and interact with them programmatically. I have tested two websites:

- Officeworks registration page
- Demoblaze signup page(A fake/clone random website that chatGPT suggested me for easy login and signup details)



2. Selenium WebDriver Locator APIs (in simple terms)

Locator APIs are used to identfy elements on a webpage. Since Selenium cannot “see” elements like humans, we need to tell it exactly which element to interact with by finding firstly an inspect element within the browser.

In my program, I have used the following locators:

➤ By.id()

This is the most reliable and fastest method. If an element has a unique ID, it is always better to use this.

Example:

driver.findElement(By.id("firstname"));

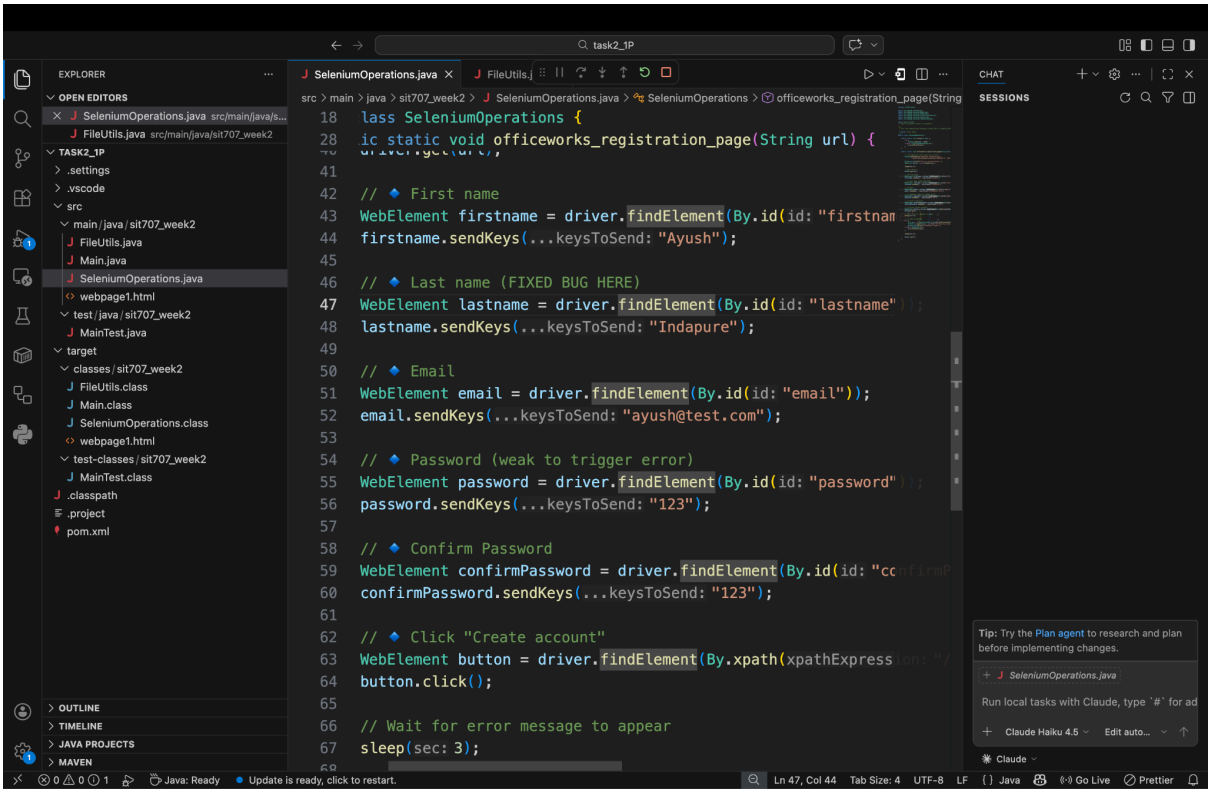
➤ By.xpath()

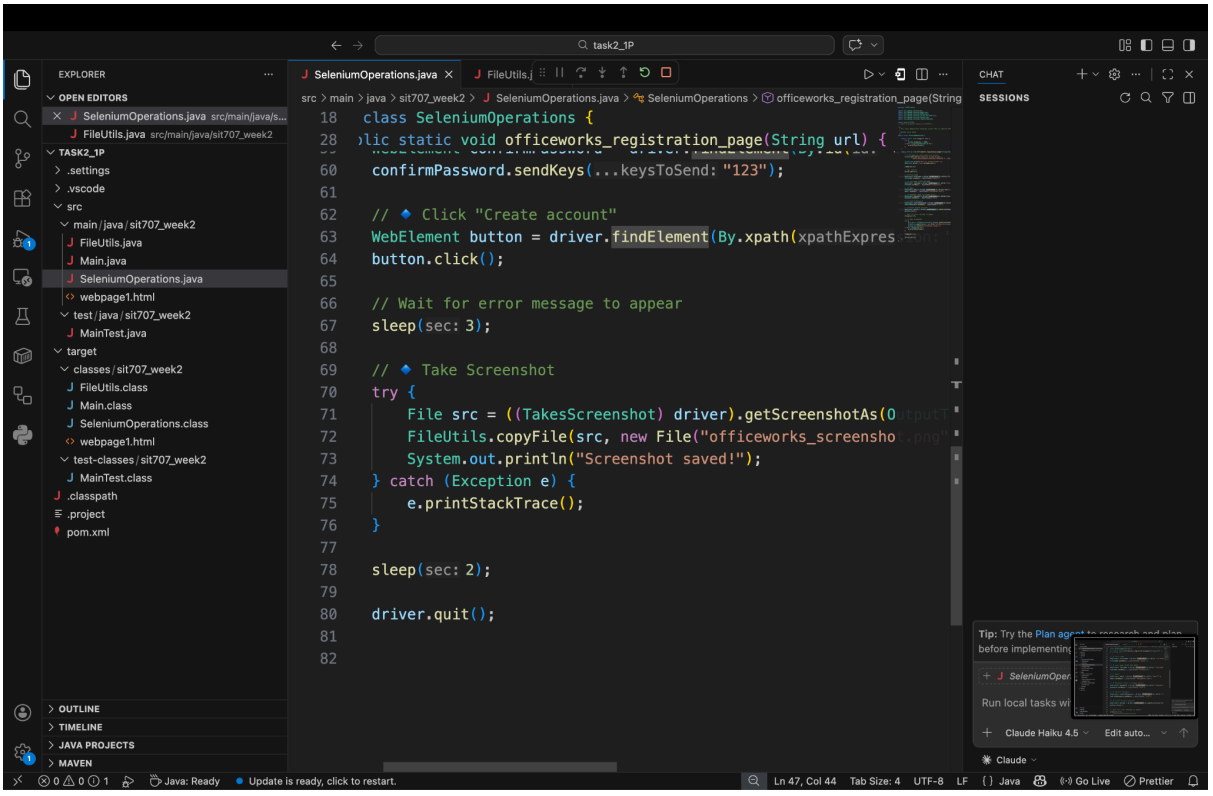
XPath is used when elements do not have proper IDs. It allows us to locate elements based on their structure or text.

Example:

driver.findElement(By.xpath("//button[contains(text(),'Create account')]"));

👉 In simple words, first we inspect the webpage manually, check the HTML structure, and then decide which locator is best to use.


task2_1P
EXPLORER
OPEN EDITORS
SeleniumOperations.java src/main/java/s...
FileUtils.java src/main/java/sit707_week2
TASK2_1P
.settings
.vscode
src
main/java/sit707_week2
FileUtils.java
Main.java
SeleniumOperations.java
webpage1.html
test/java/sit707_week2
MainTest.java
target
classes/sit707_week2
FileUtils.class
Main.class
SeleniumOperations.class
webpage1.html
test-classes/sit707_week2
MainTest.class
.classpath
.project
pom.xml
OUTLINE
TIMELINE
JAVA PROJECTS
MAVEN
SeleniumOperations.java
FileUtils.j
src > main > java > sit707_week2 > SeleniumOperations.java > SeleniumOperations > officeworks_registration_page(String
lass SeleniumOperations {
ic static void officeworks_registration_page(String url) {
// 🔹 First name
WebElement firstname = driver.findElement(By.id(id: "firstnam
firstname.sendKeys(...keysToSend: "Ayush");
// 🔹 Last name (FIXED BUG HERE)
WebElement lastname = driver.findElement(By.id(id: "lastname"
lastname.sendKeys(...keysToSend: "Indapure");
// 🔹 Email
WebElement email = driver.findElement(By.id(id: "email"));
email.sendKeys(...keysToSend: "ayush@test.com");
// 🔹 Password (weak to trigger error)
WebElement password = driver.findElement(By.id(id: "password"
password.sendKeys(...keysToSend: "123");
// 🔹 Confirm Password
WebElement confirmPassword = driver.findElement(By.id(id: "cc
confirmPassword.sendKeys(...keysToSend: "123");
// 🔹 Click "Create account"
WebElement button = driver.findElement(By.xpath(xpathExpress
button.click();
// Wait for error message to appear
sleep(sec: 3);
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
SeleniumOperations.java
Run local tasks with Claude, type `#` for ad
Claude Haiku 4.5
Edit auto...
Claude
Java: Ready
Update is ready, click to restart.
Ln 47, Col 44
Tab Size: 4
UTF-8
LF
Java
Go Live
Prettier


task2_1P
EXPLORER
OPEN EDITORS
SeleniumOperations.java src/main/java/s...
FileUtils.java src/main/java/sit707_week2
TASK2_1P
.settings
.vscode
src
main/java/sit707_week2
FileUtils.java
Main.java
SeleniumOperations.java
webpage1.html
test/java/sit707_week2
MainTest.java
target
classes/sit707_week2
FileUtils.class
Main.class
SeleniumOperations.class
webpage1.html
test-classes/sit707_week2
MainTest.class
.classpath
.project
pom.xml
OUTLINE
TIMELINE
JAVA PROJECTS
MAVEN
SeleniumOperations.java
FileUtils.j
src > main > java > sit707_week2 > SeleniumOperations.java > SeleniumOperations > officeworks_registration_page(String
class SeleniumOperations {
blic static void officeworks_registration_page(String url) {
confirmPassword.sendKeys(...keysToSend: "123");
// 🔹 Click "Create account"
WebElement button = driver.findElement(By.xpath(xpathExpres
button.click();
// Wait for error message to appear
sleep(sec: 3);
// 🔹 Take Screenshot
try {
File src = ((TakesScreenshot) driver).getScreenshotAs(O
FileUtils.copyFile(src, new File("officeworks_screensho
System.out.println("Screenshot saved!");
} catch (Exception e) {
e.printStackTrace();
}
sleep(sec: 2);
driver.quit();
CHAT
SESSIONS
Claude Haiku 4.5
Edit auto...
Claude
Java: Ready
Update is ready, click to restart.
Ln 47, Col 44
Tab Size: 4
UTF-8
LF
Java
Go Live
Prettier

3. Selenium Element Interaction APIs (in simple terms)

Once the element is identified, we need to interact with it like a real user.

➤ sendKeys()

This is used to type text into input fields.

Example:

element.sendKeys("Ayush");
(and likewise)

➤ click()

This is used to click buttons or links.

Example:

element.click();

👉 These methods help us simulate real user behaviour like filling forms and clicking submit buttons.

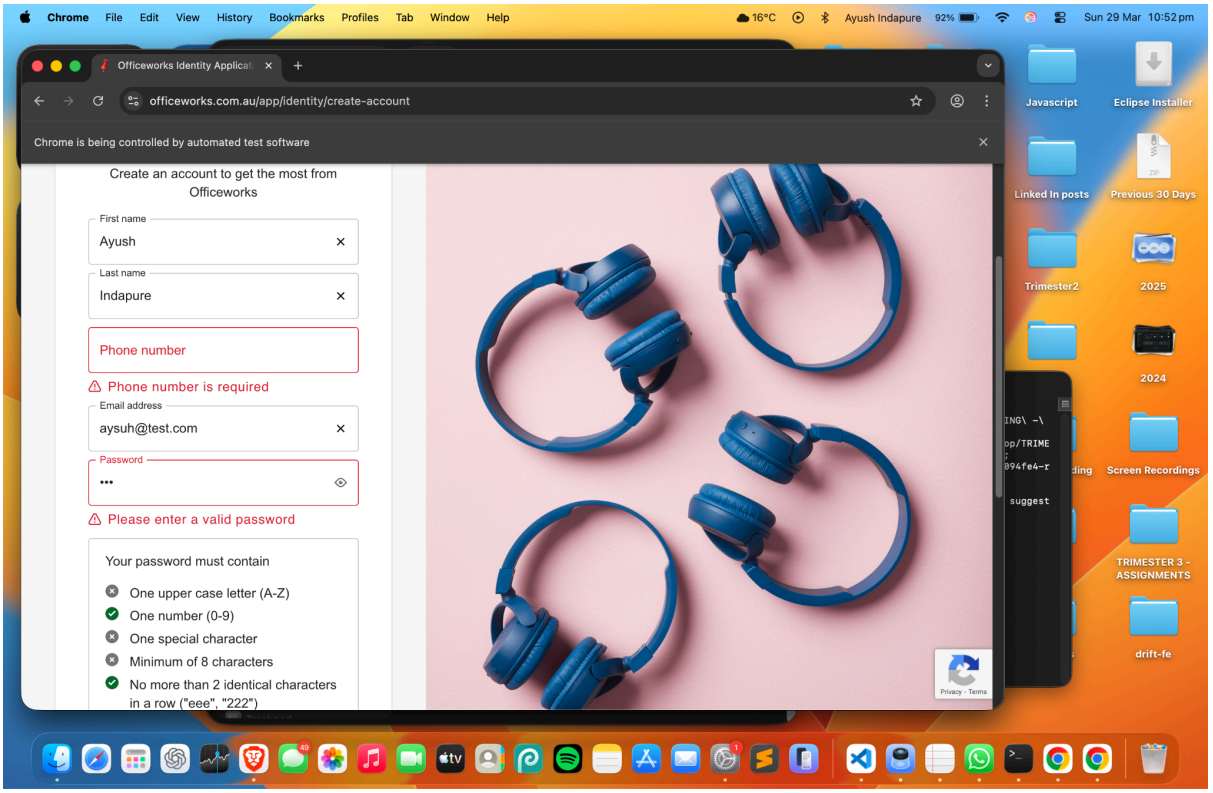
---

4. Testing Officeworks Registration Page

In this part, I automated the registration process on the Officeworks website.

Steps performed:

- Opened the registration page
- Located all input fields using `By.id()`
- Entered personal details
- Used a weak password intentionally to trigger validation error
- Clicked on “Create account” button
- Captured screenshot of the error state



👉 Observation:
The website had proper IDs for most elements, which made it easier to automate. However, the submit button required XPath.

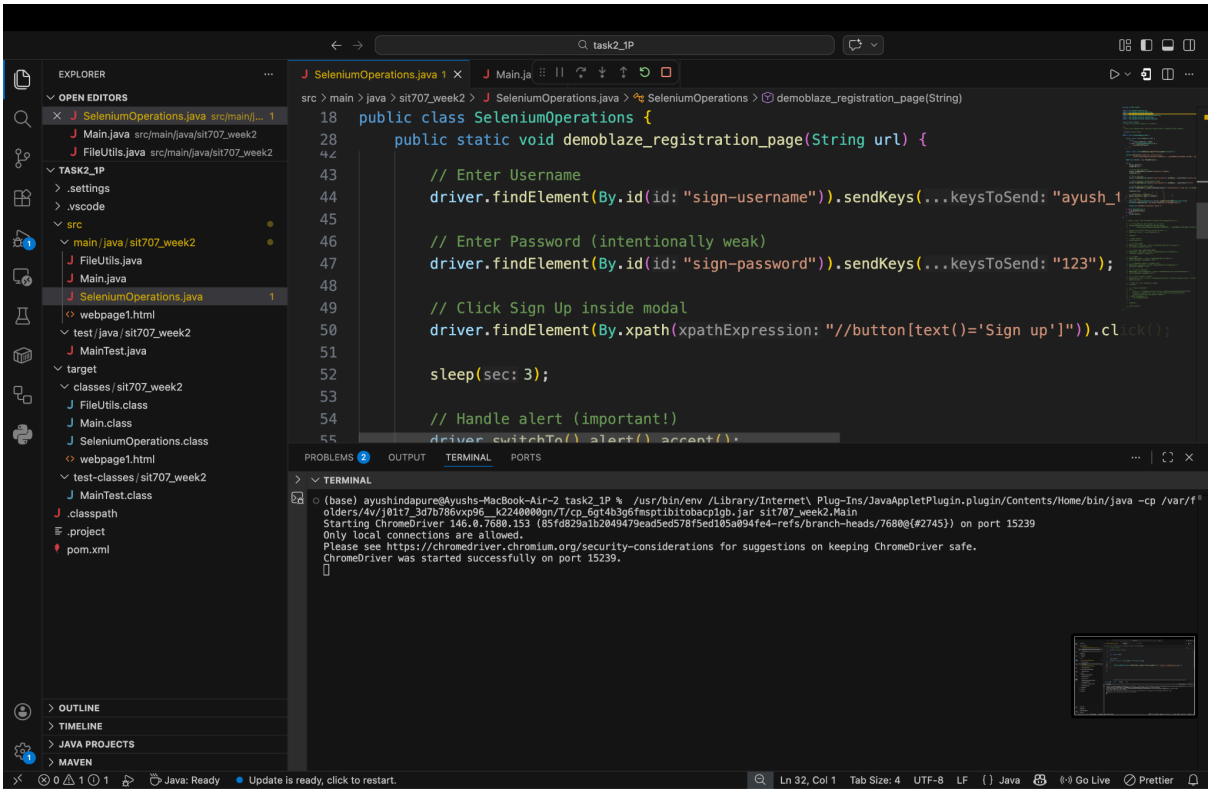
---

5. Testing Demoblaze(fake website suggested by chatGPT for education purpose) Registration Page

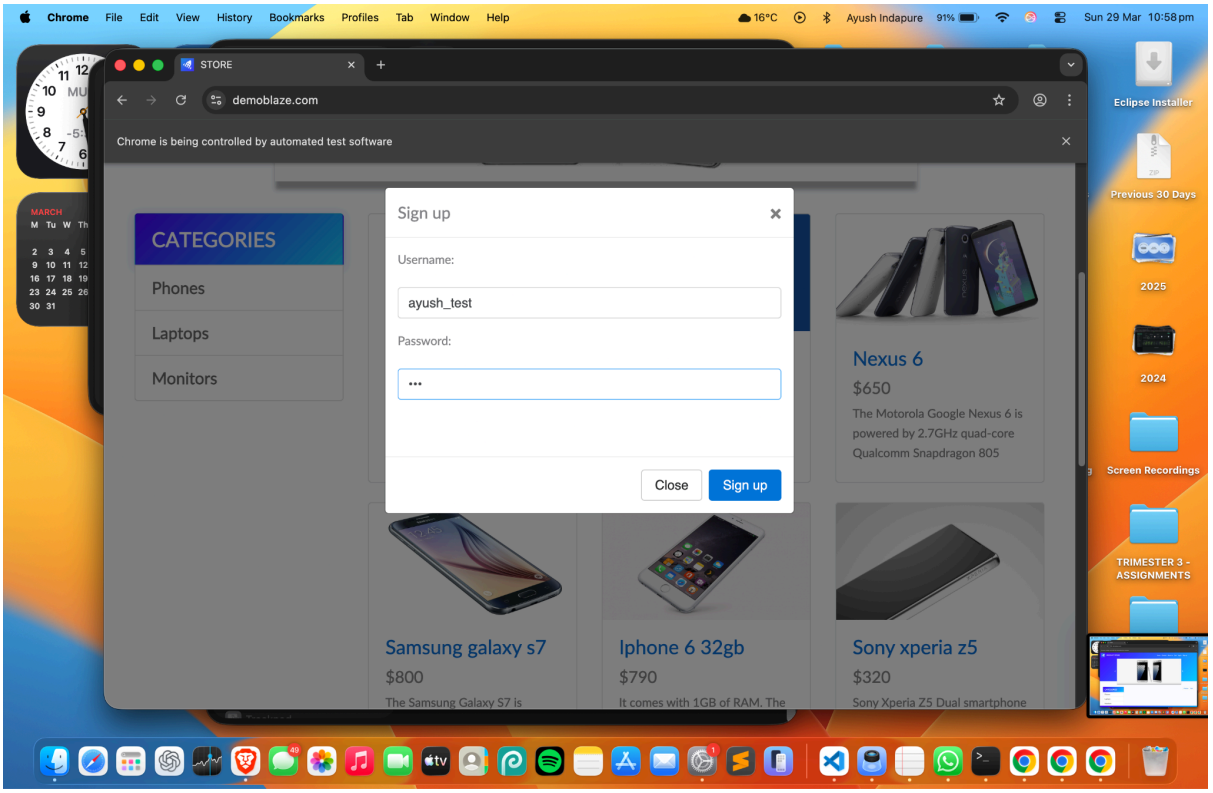
In this part, I automated the signup process on Demoblaze.

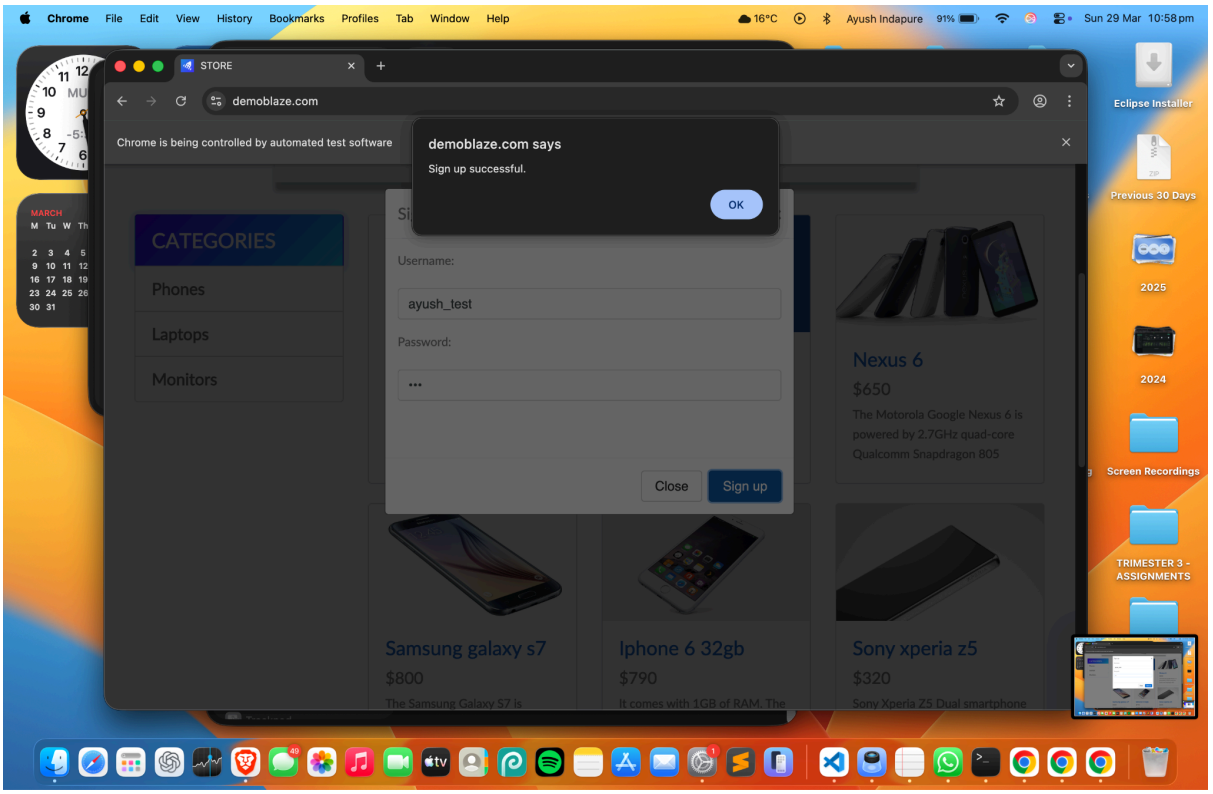
Steps performed:

- Opened homepage
- Clicked on “Sign up” button
- Entered username and password
- Clicked signup button
- Handled alert popup
- Captured screenshot



👉 Observation:
This website uses a popup (modal) instead of a full page form, so it was slightly different to handle. Also, after submission, an alert appears which needs to be handled separately.





---

6. Experience and Comparison

While working with both websites, I noticed some key differences:

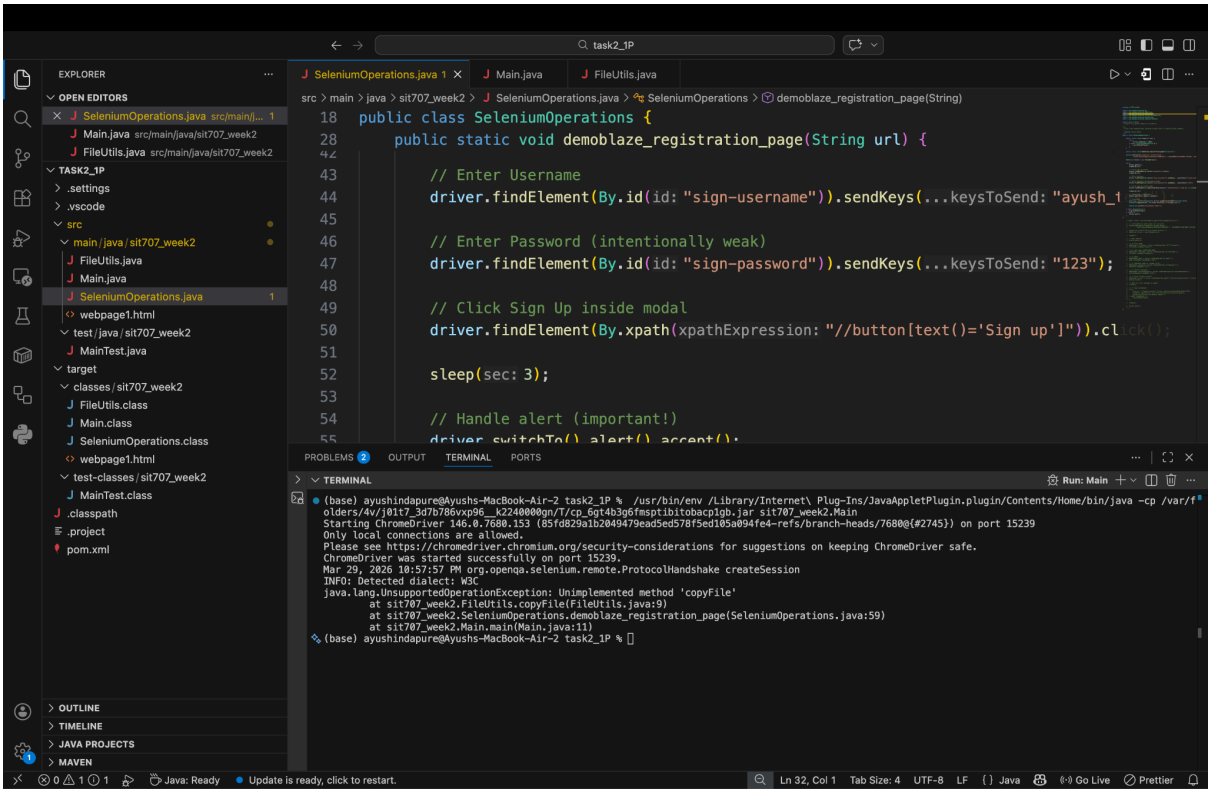
- Officeworks has a structured registration form with clear IDs, which made element identification easier.
- Demoblaze uses a modal popup, which required additional steps like clicking a button to open the form.
- Officeworks required XPath for the submit button, whereas Demoblaze had simpler locators.
- Demoblaze also required handling alert popups after submission, which is not present in Officeworks.

Overall, Officeworks was easier for basic form automation, while Demoblaze provided a more dynamic scenario with popups and alerts.

---

7. Challenges Faced

- Initially faced issues with ChromeDriver setup on Mac
- Some elements did not have clear IDs, so XPath had to be used
- Handling alert popup in Demoblaze required additional steps
- Managing page load timing using sleep



---

8. Key Learnings

- Importance of choosing the right locator for stable automation
- One of the disadvantage of selenium is its quite slow and old-fashioned.
- Lots of new e2e technologies has come into picture one of the many example is cypress
- Cypress has truly taken a hold due to its ability to do everything on its own without manual human interventsion
- Understanding how different websites structure their forms
- Learning how to handle dynamic elements like popups and alerts
- Selenium helps in automating real user actions effectively

---

9. Conclusion

This task helped me understand how Selenium WebDriver works in real-world scenarios. I learned how to locate elements, interact with them, and handle different types of web interfaces. This knowledge will be useful for automated testing and improving software quality.

---

10. GitHub Submission

The project has been uploaded to GitHub as a private repository. A screenshot of the repository is attached.

Live link to the repo : https://github.com/ayushindapure/sit707-2.1P